

This course has already ended.

« Chapter 16.2: More about text file formats (https://plus.cs.aalto.fi/studio_2/k2022/w16/) / Round 17 » (https://plus.cs.aalto.fi/studio_2/k2022/w17/)
CS-C2120 (https://plus.cs.aalto.fi/studio_2/k2022/) / Round 16 (https://plus.cs.aalto.fi/studio_2/k2022/w16/)
/ 16.3 File IO: Reading JSON file

16.3 File IO: Reading JSON file

About this page:

Key questions: How to use libraries to read JSON?

Topics: JSON and the Circe library.

What will I do? Read about JSON and Circe.

Indicative difficulty level: Easy

Rough estimate of workload? 0.5 hours

About JSON

JSON is a human-readable text-based data interchange format, that is widely used in computer science. The format name is an abbreviation of JavaScript Object Notation, even though the format is independent of JavaScript. The following contains one example of using JSON with Scala, as it is likely that you will run into the format sooner or later.

Here is the JSON data we will be working with:

```
[
{
  "id": "5ef0a9a92359ffa63a9cb56e",
  "age": 25,
  "name": "Georgette Tillman",
  "gender": "female",
  "pets": [
    {
      "age": 4,
      "name": "Gibson"
    },
    {
      "age": 2,
      "name": "Mr. Harrington"
    }
  ]
},
{
  "id": "5ef0a9a946a00ec035b75f09",
  "age": 22,
  "name": "Duffy Fulton",
  "gender": "male",
  "pets": [
    {
      "age": 1,
      "name": "Mills"
    }
  ]
}
]
```

This example data contains a JSON list object with two other JSON objects inside. These objects represent people who have pets.

A JSON object consists of curly brackets and key-value pairs {"Key": value} . Keys are strings. Values can be strings, numbers, booleans, lists, and even other JSON objects. In our data, the Person objects have a list of Pet objects.

The key-value pairs are separated from each other by commas , . A list is denoted with square brackets []. Objects inside a list are separated with a comma , .

Parsing JSON data

We will convert this JSON data into a form that is usable by our program. In this case, our goal is to convert the data into a list of Person objects in Scala. This process is known as parsing.

Several Scala libraries offer tools for parsing JSON data. Some examples of them and their use are presented here: <https://manuel.bernhardt.io/2015/11/06/a-quick-tour-of-json-libraries-in-scala/>  (https://manuel.bernhardt.io/2015/11/06/a-quick-tour-of-json-libraries-in-scala/)

In this example we will be using Circe, as it has fairly simple but powerful tools for our purposes. Circe offers different alternatives for parsing JSON. We can parse it to a JObject, or use the library's derivation capabilities to receive our own object types. Circe knows how to convert JSON to objects and vice versa via decoders and encoders.

In this example we use automatic derivation where Circe figures out the decoders and encoders based on case classes we define, but semi-automatic derivation where JSON decoders and encoders are defined implicitly by the programmer is also possible.

Let's get to parsing!

We will need to import the following:

```
import io.circe.generic.auto._, io.circe.syntax._, io.circe.parser.decode
```

First, we need to define case classes Person and Pet. These case classes convey the form or schema of our JSON data to Circe. Notice how nested data is handled.

```
case class Person(id: String, name: String, age: Int, gender: String, pets: List[Pet])
case class Pet(name: String, age: Int)
```

Circe can read JSON data in string form. If our JSON was in a file, we would read it using normal FileIO tools. This time, we will use scala.io.source like this:

```
val fileSource = util.Try(scala.io.Source.fromFile("foo.bar"))
val stringSource = util.Try(scala.io.Source.fromString(jsonString))

def readText(source: scala.io.Source) = source.getLines().mkString("\n")
```

Try is used, as the Source methods may throw exceptions if the source is not found.

Once we have our JSON data in string form, parsing becomes a matter of calling Circe's decode method with the wanted result type as a type parameter. Circe then returns the parsing result in an Either, which is a similar structure to Try. In fact, an Either can be converted to a Try with the method toTry.

```
def readJson(text: String) = decode[List[Person]](text).toTry
```

Finally, we create the list of people.

```
val people = {  
  
    val temp = Buffer[Person]()  
    for {  
        source <- stringSource    // Get succesful source result  
        text   = readText(source) // Get text from source  
        list   <- readJson(text)  // Parse JSON into list  
        person <- list  
    } temp += person  
  
    temp  
}
```

Here we can see the power of for comprehensions. They allow us to extract the successful result from the Try and use it for further operations. If a failure occurs in one of the Trys, the next loop generator lines are not executed.

Our result:

```
ArrayBuffer(  
  Person(5ef0a9a92359ffa63a9cb56e,Georgette Tillman,25,female,List(Pet(Gibson,4), Pet(Mr. Harrington,2))),  
  Person(5ef0a9a946a00ec035b75f09,Duffy Fulton,22,male,List(Pet(Mills,1)))
```

Complete code example

```

import scala.collection.mutable.Buffer
import scala.util.Try
import java.io.FileReader
import scala.util.Failure
import java.io.FileNotFoundException
import scala.util.Success
import java.io.BufferedReader

import io.circe.generic.auto._, io.circe.syntax._, io.circe.parser.decode

object JsonExample extends App {

  val jsonString = """
  [
  {
    "id":"5ef0a9a92359ffa63a9cb56e",
    "age":25,
    "name":"Georgette Tillman",
    "gender":"female",
    "pets":[
      {
        "age":4,
        "name":"Gibson"
      },
      {
        "age":2,
        "name":"Mr. Harrington"
      }
    ]
  },
  {
    "id":"5ef0a9a946a00ec035b75f09",
    "age":22,
    "name":"Duffy Fulton",
    "gender":"male",
    "pets":[
      {
        "age":1,
        "name":"Mills"
      }
    ]
  }
  ]
  """

  /**Circe is able to use the following definitions to automatically
  decode correctly formatted JSON*/
  case class Person(id: String, name: String, age: Int, gender: String, pets: List[Pet])
  case class Pet(name: String, age: Int)

  /** Reads in a JSON formatted string as a List if possible*/
  def readJson(text: String ) = decode[List[Person]](text).toTry

  /** Reads all the contents of a given Source as a String */
  def readText(source: scala.io.Source ) = source.getLines().mkString("\n")

  /** Note: In larger programs, source would be closed manually */
  val fileSource  = util.Try(scala.io.Source.fromFile("foo.bar"))
  val stringSource = util.Try(scala.io.Source.fromString(jsonString))

  val people = {

```

```
val temp = Buffer[Person]()

for {
  source <- stringSource    // Get successful source result
  text   = readText(source) //Get text from source
  list   <- readJson(text)  //Parse JSON into list
  person <- list
} temp += person

temp

println(people)

}
```
